

iRUC: Reducing Inter-Microservice Data Communication in Data-Intensive Systems via Unified Computation

Puwei Wang , Ruiheng Liu , Keman Huang , and Xiaoyong Du , *Member, IEEE*

Abstract—In data-intensive microservice-based systems, frequent and large-scale inter-service communication poses a critical performance bottleneck, degrading throughput and escalating latency. Existing solutions exhibit notable limitations: microservice merging sacrifices loose coupling and system evolvability; resource-aware scheduling enhances communication efficiency but fails to reduce data volume; dynamic deployment reduces network distances while introducing compute-resource contention; and unnecessary data transfer elimination remains ineffective under massive data loads. Hence, to overcome these challenges, we propose iRUC, an approach for inter-service data communication Reduction via Unified Computation. In particular, we first design *GraphQL+*, an executable declarative language that extends GraphQL with service-interaction semantics to achieve unified, cross-language modeling of data processing and transmission across microservices. Second, we develop an *LLM-based multi-agent system* leveraging Claude 4.5 Sonnet and Gemini 2.5 Pro to automatically parse microservice code and synthesize corresponding GraphQL+ models. Third, we implement the unified *execution engine* for GraphQL+ models, including the database gateway that preserves microservice database autonomy while enabling cross-database queries. This design enables iRUC to perform the unified modeling and execution of data processing and transmission across microservices, thereby significantly reducing inter-service data transfer while maintaining microservice independence. Experimental evaluation on nine GitHub open-source microservice projects deployed on Huawei Cloud demonstrates iRUC's effectiveness: compared with the unnecessary transfer elimination, dynamic deployment, and serverless computing approaches, iRUC improves throughput by $5.57\times$, $1.52\times$, and $1.87\times$, respectively, while reducing latency to 7.7%, 40.7%, and 37.4% of those approaches. These results show that iRUC achieves significant performance improvements in large-scale data processing scenarios.

Index Terms—Inter-Service data communication, GraphQL+, LLM-based multi-agent system, unified computation.

I. INTRODUCTION

THE proliferation of large-scale data generation and its strategic importance have catalyzed the development of data-intensive systems, which are engineered to extract insights from massive and dynamic datasets [1], [2]. As these systems scale, managing the complexity arising from both application logic and data processing pipelines has become increasingly important [3]. To improve modularity, scalability, and service-level reusability, microservice architecture has emerged as a widely adopted design paradigm [4]. However, this architectural shift introduces additional runtime challenges, particularly the overhead associated with inter-microservice data communication, which is amplified in data-intensive systems [5], [6]. In such systems, frequent and high-volume data exchange among microservices can markedly degrade throughput and increase bandwidth usage [7]. These observations highlight the importance of exploring more efficient inter-service communication approaches, especially in scenarios where data transfer constitutes a primary performance bottleneck [8].

To address the data-communication bottleneck between microservices, existing research has primarily focused on three optimization strategies. First, the service granularity design strategy [9], [10] merges microservices with high-frequency interactions based on data-transfer overhead to eliminate runtime cross-microservice data transfer. But, these approaches partially compromise the loose-coupling principle: merged microservices often differ significantly in business logic and lifecycle management, while merging them increases system complexity, reduces independent evolvability and maintainability, and undermines the agility needed to respond quickly to user requirements. Second, the resource-aware scheduling strategy [11], [12] optimizes communication efficiency by predicting service load and dynamically adjusting CPU, memory, and network-bandwidth allocations. Although such approaches can enhance communication efficiency to some extent, they increase resource consumption and do not fundamentally resolve the root cause of inter-service data transfer. Third, the dynamic deployment strategy [13], [14], [15] co-locates frequently interacting microservices on the same node to shorten network paths and lower latency. But,

Received 16 May 2025; revised 31 December 2025; accepted 15 January 2026. Date of publication 23 January 2026; date of current version 20 April 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB2702102 and in part by the National Natural Science Foundation of China under Grant 62172425, Grant 62572474, Grant 62436010, and Grant 62441230. Recommended for acceptance by S. McIntosh. (Corresponding author: Keman Huang.)

The authors are with the School of Information and with the Key Laboratory of Data Engineering and Knowledge Engineering of the Ministry of Education, Renmin University of China, Beijing 100872, China (e-mail: wangpuwei@ruc.edu.cn; liuruiheng@ruc.edu.cn; keman@ruc.edu.cn; duyong@ruc.edu.cn).

Digital Object Identifier 10.1109/TSE.2026.3656819

beyond increasing the complexity of system maintenance, large-scale centralized deployments are constrained by compute resources: when many services share a single server, contention for CPU and memory can lead to performance degradation.

Hence, while existing strategies offer promising solutions for enhancing the communication performance of microservice-based data-intensive systems, they can only alleviate data-communication bottlenecks to a certain extent. They fall short of fundamentally reducing the volume of data exchanged between microservices, and thus cannot achieve a substantive improvement in overall system performance. Recent studies begin to focus on the data transfer load itself, seeking to reduce transmission volumes at its source. Genfer et al. [16] highlight that problematic service interfaces or unclear service responsibilities can lead to the exchange of unnecessary data, exacerbating the data communication burden. Tariq et al. [17] further propose MicroProf, a dynamic program analysis tool for Java-based microservice applications, to detect and identify unnecessary data transfers. However, in scenarios involving large volumes of data, the inter-service data transmission remains a performance bottleneck. Therefore, in data-intensive systems, it is not sufficient to focus only on “*how data is transmitted*”; it is important to explore the issue of “*how to minimize the data that needs to be transmitted*”.

This motivates us to explore a new strategy that can minimize inter-service data transmission in microservice-based data-intensive systems. Particularly, in these systems, data transmission and processing are typically fragmented and distributed across different microservices, resulting in excessive inter-service communication overhead. To address this issue, we propose iRUC, an approach for inter-service data communication Reduction via Unified Computation. The core of iRUC is the *unified computation*: uniformly modeling data processing and transmission across microservices and executing them centrally through a dedicated engine.

Building on this idea, for data processing and transmission, we design GraphQL+, a declarative executable language based on GraphQL [18], supporting fine-grained and flexible expression of the data-related semantics. Additionally, to ensure compatibility with loosely coupled microservice systems, we develop a large language model (LLM)-based multi-agent system. Leveraging LLMs’ advanced understanding of programming languages [19], [20], [21], the agent system decomposes the modeling tasks into a set of subtasks and assigns them to specialized agents for collaborative processing. It automatically analyzes microservice source code to identify data processing and transmission, generating corresponding GraphQL+ models. Upon service code updates, the agent system synchronizes the GraphQL+ models, thereby preserving the independent maintenance and deployment capabilities of microservices. Furthermore, for the generated GraphQL+ models, we implement an execution engine integrated with a database gateway. The engine merges the GraphQL+ models generated from microservices into a composite model and then centrally executes it, while the gateway provides a data access layer that preserves the autonomy of individual microservice databases and enables

seamless cross-database queries. The iRUC approach uniformly models data processing and transmission across microservices, then moves them to a database-adjacent node for centralized execution, thereby eliminating inter-service data communication and substantially improving system performance.

To validate the effectiveness of our approach, we select nine open-source microservice systems from GitHub as our experimental subjects. For the LLM-based multi-agent systems, we employ Claude 4.5 Sonnet API to automatically generate GraphQL+ models and use Gemini 2.5 Pro for functionality checking, with tests confirming their correctness. All systems are deployed on Huawei Cloud to replicate real-world conditions. We compare our approach against three baselines: the excessive data-transfer removal approach (ER), the dynamic deployment approach (DD) and the serverless computing approach (SC). On average, the iRUC approach achieves $5.57\times$ the throughput of ER, $1.52\times$ that of DD and $1.87\times$ that of SC. In terms of latency, it reduces latency to only 7.7% of that of ER, 40.7% of that of DD and 37.4% of that of SC.

In summary, we make contributions as follows:

- We design GraphQL+, a declarative executable language that provides unified cross-language modeling of data processing and transmission across microservices;
- We develop an LLM-based multi-agent system that generates and synchronizes GraphQL+ models from microservice code, preserving compatibility with loosely coupled microservice architectures;
- We implement a GraphQL+ execution engine to efficiently parse and execute the GraphQL+ models and design a unified database gateway that supports transparent cross-database queries while preserving individual database autonomy;
- We select nine popular GitHub microservice systems and use Claude 4.5 Sonnet and Gemini 2.5 Pro to automatically generate and check GraphQL+ models. We deploy these systems on Huawei Cloud, and the experimental results demonstrate significant performance improvements in throughput and latency, especially in large-scale data scenarios.

The remainder of this paper is organized as follows. Section II presents a motivation case. Section III describes the design of iRUC. Section IV presents experimental results based on nine open-source microservice projects from GitHub. Section V reviews the related work. Section VI discusses potential threats to validity and outlines directions for future research and practice. Finally, Section VII concludes the paper.

II. MOTIVATION CASE

To better illustrate our approach, we use the open-source microservice system go-zero-look (<https://github.com/Mikaelemmmm/go-zero-looklook>) as a motivating example. This system has two microservices: order-API and payment-RPC, which collaborate to retrieve order information and third-party payment details. Fig. 1 presents a simplified version of the relevant code. The order-API service first retrieves order

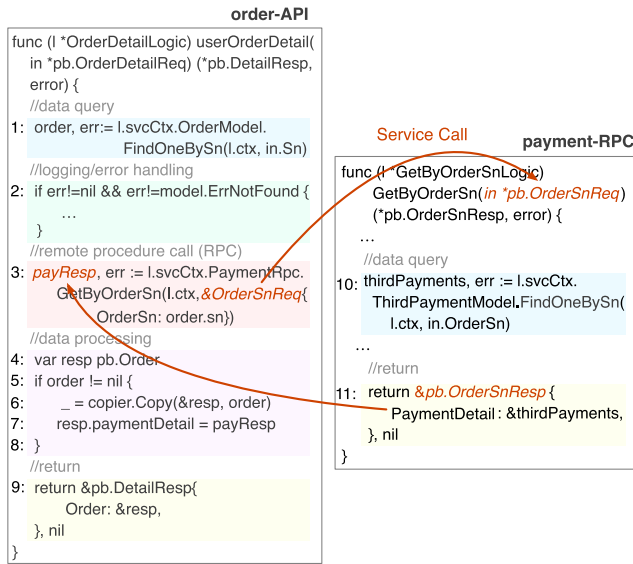


Fig. 1. The service call between order-API and payment-RPC services (<https://github.com/mikaelemmmm/go-zero-looklook/tree/main>).

details from its database using the serial number (Line 1) and handles any errors that occur (Line 2). It then sends an RPC request to the payment-RPC service, passing the serial number as input (Line 3). The payment-RPC service then queries its database for the corresponding third-party payment details (Line 10) and returns them to the order-API service (Line 11). Upon receiving the response, the order-API service copies the order details into a response object and assigns the retrieved payment details to it (Lines 4–8). Finally, the DetailResp object containing the order and payment details is returned to the client (Line 9).

As this microservice system scales with more users, the volume of transferred data grows rapidly, leading to massive network data transfer. This results in a sharp increase in latency and a substantial drop in throughput. To evaluate the impact of data transfer, we deploy this microservice system on Huawei Cloud. Experimental results show that when the data size per request exceeds 1MB, the response latency surpasses 1 second. Especially, when the data size reaches 6MB, the latency exceeds 5 seconds, severely degrading user experience. Breakdown analysis reveals that inter-service data transmission accounts for about 80% of the total latency, making it the primary performance bottleneck.

Hence, in this study, our goal is to revisit the microservice architecture for data-intensive systems to resolve the inter-service data communication overhead. Using the order-API service code in Fig. 1 as an example, there is a function `userOrderDetail` which involves data processing and transmission with the payment-RPC service. We can categorize the code into five types based on how they manage the related data:

- (1) *Data Query* (Line 1): Operations for creating, reading, updating, and deleting data in the database, typically managed by persistence frameworks such as JPA or MyBatis;

- (2) *Logging/Error Handling* (Line 2): Capturing logs and handling exceptions to improve system observability and ensure operational stability;
- (3) *Remote Procedure Call (RPC)* (Line 3): Inter-service communication, typically implemented via protocols like gRPC or RESTful APIs;
- (4) *Data Processing* (Lines 4–8): Data computations or transformations that do not involve persistent storage access or remote service calls;
- (5) *Return* (Line 9): Returns the processed result to the clients or other services, typically as the service response.

In these five types of code, our focus is on data query, data processing, remote procedure call and return handling, as they are related to data transmission across microservices. For data query, GraphQL [18] provides a flexible and efficient way to access data. It allows clients to request only the data they need, avoiding the issues of over-fetching (retrieving unnecessary data) or under-fetching (not retrieving enough data). For its fine-grained data querying abilities, GraphQL allows multiple resources to be fetched in a single request, eliminating the need for multiple round trips to the database server. This feature optimizes database access and minimizes the overhead of network communication. However, as shown in Fig. 1, the order-API service code includes more than just data query, and it also involves service call and data processing.

To this end, inspired by the design principle of GraphQL, we design GraphQL+, a language-agnostic declarative executable language built on the top of GraphQL, to unify the data query, processing and transmission between microservices. Additionally, we develop the iRUC approach to abstract the data processing and transmission between microservices into the GraphQL+ models, which are subsequently deployed on a database-adjacent node for unified computation by a dedicated engine. As this approach enables the unified modeling and execution of data processing and transmission across microservices, it effectively eliminates the inter-service data transfer and enhances overall system performance.

III. SYSTEM DESIGN

As illustrated in Fig. 2, we propose iRUC, an approach for inter-service data communication Reduction via Unified Computation, aiming at eliminating the inter-service data transfer among microservices. Microservices feature a loosely coupled architecture, supporting independent deployment and maintenance while continuously adapting to dynamic business requirements through iterative updates. Microservice operations can be categorized into two types: *update* (code modification) and *execution* (handling user requests). For update operations, when the microservice code changes, the iRUC approach synchronously updates its corresponding GraphQL+ model. To automate the generation and updating of models, we develop a multi-agent system based on large language models (LLMs) that analyzes microservice code to generate or update the GraphQL+ models. The GraphQL+ models of data-transfer-related microservices are then composed into a composite model,

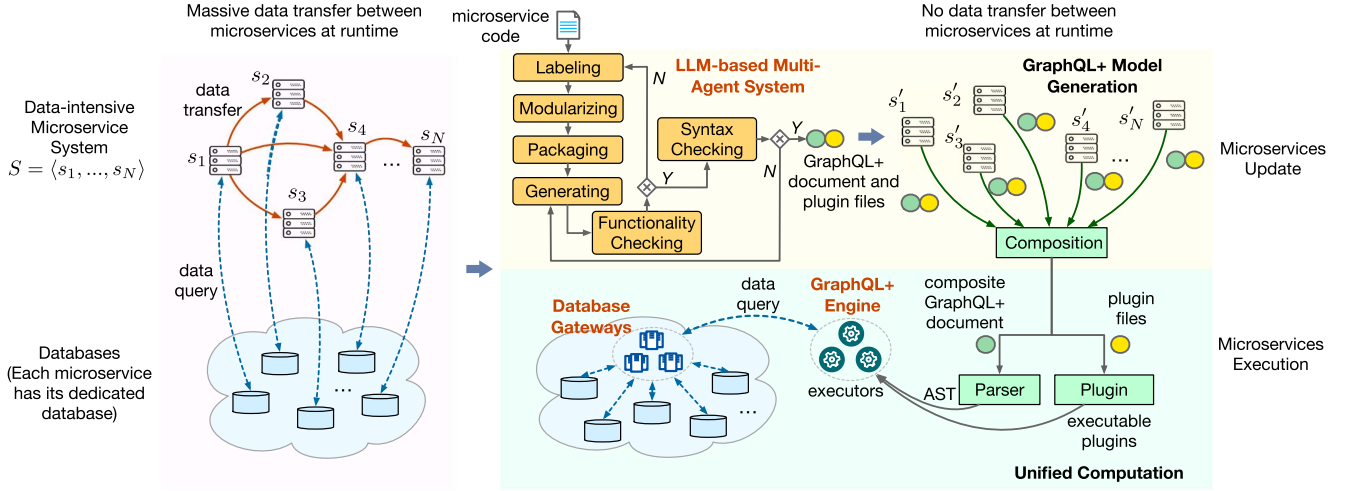


Fig. 2. iRUC: An approach for inter-service data communication overhead reduction through unified computation.

```

document      ← statement*
statement     ← dataQueryStmt
               / assignStmt
               / serviceCallStmt
               / pluginCallStmt
               / returnStmt

dataQueryStmt ← "new" variable "=" "gql" ( "query" / "mutation" ) "{" gqlBody "}" ";"
assignStmt    ← "assign" globalVariable "=" variable ";"
               / "assign" variable "=" globalVariable ";"

globalVariable ← service "." variable

serviceCallStmt ← assignStmt*
                 remoteCallStmt
                 assignStmt*

remoteCallStmt ← "call" service ";" ;
pluginCallStmt ← "new" variableList "=" "plugin" file "/" function "(" variableList ")" ";" ;
variableList   ← variable ( "," variable ) *
returnStmt     ← assignStmt*
                 terminateStmt
terminateStmt  ← "return" globalVariable ";" ;
...

```

Fig. 3. GraphQL+ Specification in PEG.

preserving the microservices' capability for independent deployment and maintenance. When large volumes of inter-data communication occur, the microservice execution operations usually face heavy load pressure. To address this issue, iRUC adopts a unified computation paradigm that executes the composite GraphQL+ model centrally, effectively eliminating the overhead caused by inter-service data transfer.

In Section III-A, we design GraphQL+, a formally defined, declarative and executable language, enabling the unified model for data processing and transmission across microservices. In Section III-B, we propose an LLM-based multi-agent system to automatically generate the GraphQL+ model from the microservice code. In Section III-C, we implement the GraphQL+ engine integrated with the database gateway.

A. GraphQL+ Specification

GraphQL+ is formally defined using PEG (Parsing Expression Grammar) [22], a parsing-expression-based formalism that ensures rigorous, implementation-friendly syntax rules. As shown in Fig. 3, GraphQL+ consists of the following types of statements:

Data Query (dataQueryStmt) The statement defines the interactions between microservices and databases. It is built upon GraphQL, with explicit variable declarations for variable tracking and management.

```
"new" variable "=" "gql" ( "query" / "mutation" ) "{" gqlBody "}" ";"
```

where the gqlBody encapsulates the GraphQL operations, and the result is subsequently assigned to the variable. The variable is a local variable, scoped only within the microservice.

For example, the query getOrderBySn(sn: Sn) can be formally described as below. It retrieves an order based on the serial number, returning the fields id, sn, description, with the result assigned to the local variable order@e2c17.

```

new order@e2c17 = gql query {
  getOrderBySn(sn: in.Sn) {
    id
    sn
    description
  }
};

```

Assignment (assignStmt) We classify variables into two types of variable scopes: local variables and global variables. Local variables are valid only within individual microservices, while global variables facilitate data transmission across services. The assignment statements define the mapping between local variables and global variables.

```

"assign" globalVariable "=" variable ";"
"assign" variable "=" globalVariable ";"

```

The first statement assigns a local variable to a global variable, and the second assigns a global variable to a local variable. A global variable can be explicitly defined using the statement globalVariable ← service "." variable, where the variable is prefixed by a service identifier (service).

For example, the local variable `order@e2c17.Sn` is assigned to the global variable `order-API.orderSnReq.OrderSn`.

```
assign order-API.OrderSnReq.OrderSn = order@e2c17.Sn;
```

Service Call (serviceCallStmt) The statement defines the interaction between microservices, including the following statements:

- *Assignment (request)*: It assigns a local variable to a global one, which is used as the request.

```
"assign" globalVariable "=" variable ","
```

where `globalVariable` $\leftarrow$ service `"."` variable represents a global variable used as the request for inter-service communication, service refers to the invoking service, and the right-hand variable represents a local variable defined within that service.

- *Remote call*: It explicitly declares the invocation relationships between services.

```
"call" service ","
```

where service is the identifier of the service to be invoked.

- *Assignment (response)*: It specifies the variable designated to hold the response returned by the invoked service.

```
"assign" variable "=" globalVariable ","
```

where `globalVariable` $\leftarrow$ service `"."` variable represents a global variable carrying the response, service refers to the invoked service, and the left-hand variable represents a local variable within the invoking service designated to store that response.

We take the following service call between the order-API and payment-RPC services as an example. The variable `order@e2c17.Sn` is assigned to the global variable `order-API.OrderSnReq.OrderSn`, which is then submitted to the payment-RPC service. After the remote invocation completes, the response stored in the global variable `payment-RPC.OrderSnResp` is assigned to the local variable `payResp@a7bf9` within the order-API service.

```
assign order-API.OrderSnReq.OrderSn = order@e2c17.Sn;
call payment-RPC;
assign payResp@a7bf9 = payment-RPC.OrderSnResp;
```

Plugin Call (pluginCallStmt) The data-processing code is often tightly coupled with business logic. We encapsulate it into self-contained, executable plugins and define corresponding call statements to enable data processing. As shown in Fig. 1, the order-API service copies the order details into an object and assigns the retrieved payment details to the object (Lines 4-8). We encapsulate the part of code into a `procOrderHandle` function and compile it into an executable plugin file as `orderDetail.so`.

The plugin call statement defines how to invoke a plugin, without involving service calls or database interactions.

```
"new" variableList "=" "plugin" file "/" function "(" variableList ")" ","
```

where file is the executable plugin file, function is the function that encapsulates the data processing logic of the service and variableList is the list of inputs and outputs of the function.

Following is an example that invokes the `procOrderHandle` function from the `orderDetail.so` plugin file. It takes the variables `order@e2c17` and `payResp@a7bf9` as the inputs, and the output is subsequently assigned to the variable `resp@9d66c`.

```
new resp@9d66c = plugin orderDetail.so/
procOrderHandle(order@e2c17, payResp@a7bf9);
```

Return (returnStmt) The statement defines how a service concludes its execution, including the following statements:

- *Assignment*: It specifies the global variable to which the return value is assigned.

```
"assign" globalVariable "=" variable ","
```

where `globalVariable` $\leftarrow$ service `"."` variable represents the global variable to which the return value is assigned, service refers to the current service, and variable represents a local variable defined within that service.

- *Terminate*: It terminates the service and returns the global variable.

```
"return" globalVariable ","
```

For example, the local variable `resp@9d66c` is assigned to the global variable `order-API.DetailResp.Order`, which is then returned as the output of the order-API service.

```
assign order-API.DetailResp.Order = resp@9d66c;
return order-API.DetailResp;
```

In summary, with GraphQL+, the data processing and transmission across microservices can be abstracted into unified, cross-language models. A GraphQL+ model includes a **declarative document** that formally describes data queries, service invocations, plugin calls, and return handling, as well as a set of **executable plugin packages** that encapsulate the data-processing logic of individual microservices.

B. LLM-Based GraphQL+ Generation

Manually developing GraphQL+ models for microservices is a non-trivial task. To automate this process, we design a large language model (LLM)-based multi-agent system. This system leverages the strengths of multiple agents, which are particularly effective when complex tasks are decomposed into smaller, manageable subtasks. Recent research works have shown that such multi-agent architectures can iteratively solve complex problems by delegating each subtask to an agent equipped with specialized capabilities [20], [21]. We adopt a conversation-based prompt engineering approach for the multi-agent system. Each agent is first presented with an illustrative example that demonstrates the desired output. Subsequently, new inputs are introduced, and the agent's preliminary outputs are iteratively refined through multi-round interactions. In each iteration, we explicitly identify shortcomings in the output and provide targeted corrective feedback. Through progressive refinement, the agent achieves stable and accurate performance. The conversation summary is then integrated into the system prompt, which, in conjunction with the task prompt, enables the agent to effectively perform its designated tasks. An example system prompt is provided in Appendix A of the supplementary materials.

As shown in Fig. 2, the LLM-based multi-agent system for generating GraphQL+ models from microservice code consists of the following sub-tasks:

Labeling: This task is to analyze microservice code using the following task prompt, aiming at achieving a deep understanding of code functionality.

#Task Description
You will be provided with the source code from a microservice project. Your task is to analyze the code and annotate five types of code blocks:
(1) Data Query: {%description%}
(2) Logging/Error Handling: {%description%}
(3) Remote Procedure Call (RPC): {%description%}
(4) Data Processing: {%description%}
(5) Return: {%description%}

You annotate the code using the following format:
//(1) Data Query
code block
//(1) end

Specifically, for data-query operations, it identifies explicit queries and infers implicit query logic expressed via ORM (Object-Relational Mapping) frameworks. For service calls, it performs semantic analysis by leveraging contextual information and parameter structures. Leveraging LLMs' cross-language understanding, the task generalizes across programming languages and coding styles.

We use the order-API code illustrated in Fig. 1 as an example. The LLM-based agent categorizes the code into five types and adds comments for distinction. The code begins by querying the database for an order record (labeled by (1)) using the provided serial number, immediately followed by error handling (labeled by (2)) that logs database errors. After retrieving the order, it constructs a request to fetch the associated payment details and makes an RPC call to the payment-RPC service (labeled by (3)). Finally, it assembles a response object including the order and payment details (labeled by (4)), and then returns it (labeled by (5)).

```
func (l *OrderDetailLogic) userOrderDetail(in *pb.OrderDetailReq)
(*pb.DetailResp, error) {
  //(1) Data Query
  order, err := l.svcCtx.OrderModel.FindOneBySn(l.ctx, in.Sn)
  //(1) end

  //(2) Logging/Error Handling
  if err != nil && err != model.ErrNotFound {
    return nil, errors.Wrapf(xerr.NewErrCode(xerr.DB_ERROR),
      "OrderModel FindOneBySn db err : %v , sn : %s", err, in.Sn)
  }
  //(2) end

  //(3) RPC
  payResp, err := l.svcCtx.PaymentRpc.GetByOrderSn(l.ctx, &OrderSnReq{
    OrderSn: order.Sn
  })
  //(3) end

  //(4) Data Processing
  var resp pb.Order
  if order != nil {
    _ = copier.Copy(&resp, order)
    resp.paymentDetail = payResp
  }
  //(4) end

  //(5) Return
  return &pb.DetailResp {
    Order: &resp,
  }, nil
  //(5) end
}
```

Modularizing: An LLM-based agent modularizes the code labeled (4) into self-contained functions with well-defined interfaces, analyzes dependencies to preserve logical coherence, and extracts the required input and output parameters. This decouples data processing from queries and service invocations, improving maintainability. The generated functions have the following characteristics: 1) Self-containment: The function operates independently, requiring no external variables. All necessary information is passed through parameters; 2) Interface clarity: The function explicitly defines its input and return parameters, ensuring clear invocation boundaries and enabling reuse; 3) High cohesion: The function focuses on the data processing, avoiding the mixing of data queries and transmission logic, thereby enhancing functional cohesion.

Hence, we design the following task prompt to lead the agent to encapsulate the code labeled by (4) into the modular functions.

#Task Description
You will be provided with the code that is categorized into 5 types. Your task is to encapsulate the code labeled by (4) into self-contained functions.

Note: The encapsulated functions should have clearly defined inputs and outputs, typically operating on objects, and return results for subsequent operations.

For example, the order-API code labeled as (4) is encapsulated into the `procOrder` function, which processes the order and `payResp` objects as input to generate a response. It has clear input and output interfaces, operates independently without external dependencies, and focuses on data processing.

```
func (l *OrderDetailLogic) procOrder(order pb.Order, payResp pb.OrderSnResp)
(*pb.Order) {
  var resp pb.Order
  if order != nil && payResp != nil {
    _ = copier.Copy(&resp, order)
    resp.paymentDetail = payResp
  }
  return resp
}
```

Packaging: This task is to encapsulate the modular functions into the dedicated plugin packages. Using the following task prompt, an LLM-based agent encapsulates the functions (e.g., `procOrder(&order, &payResp)`) into standardized plugin packages (e.g., `procOrderHandle(input []byte) ([]byte, error)`). They process JSON inputs and return JSON outputs, implementing specific data processing logic locally without relying on database operations or remote service calls.

#Task Description
You will be provided with a set of functions. Your task is to encapsulate them into dedicated plugin packages, ensuring that the packages can be compiled into executable plugins and be invoked at runtime.

Notes:

1. Each function is encapsulated as a plugin package.
2. These functions are purely computational, so you do not need to deal with any database operations and service calls.

```
func procOrderHandle(input []byte) ([]byte, error) {
  var order pb.Order
  if err := json.Unmarshal(input[0], &order); err != nil {
    return nil, fmt.Errorf("invalid input JSON: %w", err)
  }
  var payResp pb.OrderSnResp
  if err := json.Unmarshal(input[1], &payResp); err != nil {
```

```

    return nil, fmt.Errorf("invalid input JSON: %w", err)
}

detailResp, err := procOrder(&order, &payResp)
response := map[string]string{
    pluginName: "procOrder"
    order: detailResp
}

responseJSON, err := json.Marshal(response)
if err != nil {
    return nil, err
}
return responseJSON, nil
}

```

Once compiled, the plugin package becomes a binary file that can be dynamically loaded and invoked at runtime. For example, Go provides a built-in plugin environment that supports loading shared libraries (.so files) as plugins. By running the command: `go build -buildmode=plugin -o orderDetail.so`, the plugin package is compiled into an executable file `orderDetail.so`, which can be loaded and executed at runtime.

Generating: This task is to generate a GraphQL+ document from the annotated code using the following task prompt.

#Task Description

You will be provided with the annotated code. Your task is to generate a GraphQL+ document from the annotated code.

Notes:

1. For type (1), generate data query statements that describe the database operation based on its semantics;
2. For type (2), simply ignore them;
3. For type (3), generate service call statements;
4. For type (4), generate plugin call statements;
5. For type (5), generate return statements.

In particular, for the code labeled as (1) involving data queries, the agent performs semantic analysis to generate GraphQL statements, mapping data types and fields to the database schema. The code labeled as (2), such as logging and error handling, is skipped since the GraphQL+ environment provides built-in support. For the code labeled as (3), service call statements are generated; for the code labeled as (4), plugin call statements are produced, as the logic is already encapsulated in executable plugin packages. Finally, the code labeled as (5) results in the corresponding return statements.

For the order-API code shown in Fig. 1, the LLM-agent can generate the following GraphQL+ document.

```

//(1) Data Query
new order@e2c17 = gql query {
  getOrderBySn(sn: in.Sn) {
    id
    sn
    description
  }
};

//(3) RPC
assign order-API.OrderSnReq.OrderSn = order@e2c17.Sn;
call payment-RPC;
assign payResp@a7bf9 = payment-RPC.OrderSnResp;

//(4) Data Processing
new resp@9d66c = plugin orderDetail.so/
  procOrderHandle(order@e2c17, payResp@a7bf9);

//(5) Return
assign order-API.DetailResp = resp@9d66c;
return order-API.DetailResp;

```

Leveraging the collaborative efforts of these four agents

(Labeling, Modularizing, Packaging, Generating), they generate a GraphQL+ document from the code of order-API service and package the code labeled as (4) into an executable plugin file. To ensure the correctness of the generated GraphQL+ document, two additional agents are designed as follows.

Functionality Checking: This agent evaluates the equivalence between the original code and the generated GraphQL+ document. To avoid model bias and self-checking issues, we employ Claude 4.5 Sonnet for generation and Gemini 2.5 Pro for functionality checking. The checking model has no prior knowledge of the generation process and evaluates only based on the GraphQL+ syntax, enhancing the reliability of the check. This task uses the following task prompt.

#Task Description

Code: {%code%}

Document: {%GraphQL+ document%}

{%GraphQL+ Specification in EBNF%}

Your task is to evaluate the equivalence between the microservice code and the GraphQL+ document.

#Focus

1. Input-output equivalence: Ensuring the same output for the same input;
2. Boundary condition coverage: Verifying the handling of edge cases.

#Exclusions

Auxiliary code, e.g., logging, debugging information, and error handling.

Syntax Checking: Leveraging a syntax parser, this agent conducts a structured analysis of the generated GraphQL+ document, constructing an Abstract Syntax Tree (AST) and performing a syntax check. This process involves checking compliance with syntax rules, ensuring variable reference consistency, and evaluating type compatibility. The agent detects errors such as mismatched parentheses, undefined variables and incompatible types.

In summary, the LLM-based agents form a pipeline, enabling the automated step-by-step generation of GraphQL+ models from microservice code. We adopt an iterative workflow to achieve error detection and recovery. After the *Generating* agent produces a GraphQL+ model, the *Functionality Checking* agent evaluates the functional consistency between the generated model and the code. If the check fails, the feedback is submitted to the *Labeling* agent to regenerate the model. If the check passes, a *PEG parser* performs a syntax check. If the check fails, the syntax error information is fed back to the *Generating* agent until the syntax check passes.

C. GraphQL+ Engine and Database Gateway

Using the LLM-based multi-agent system, the code of each microservice involved in data transmission is abstracted into a GraphQL+ document and corresponding executable plugins. These are submitted to a dedicated engine for unified computation. As shown in Fig. 2, the GraphQL+ engine comprises four modules: *Composition*, *Plugin*, *Parser*, and *Executor*, while the database gateway provides a unified access layer for cross-service data querying.

Composition: For each microservice involved in the data transmission, we have a GraphQL+ document. For example, in the go-zero-look project, the order-API service is responsible for querying order information and obtaining

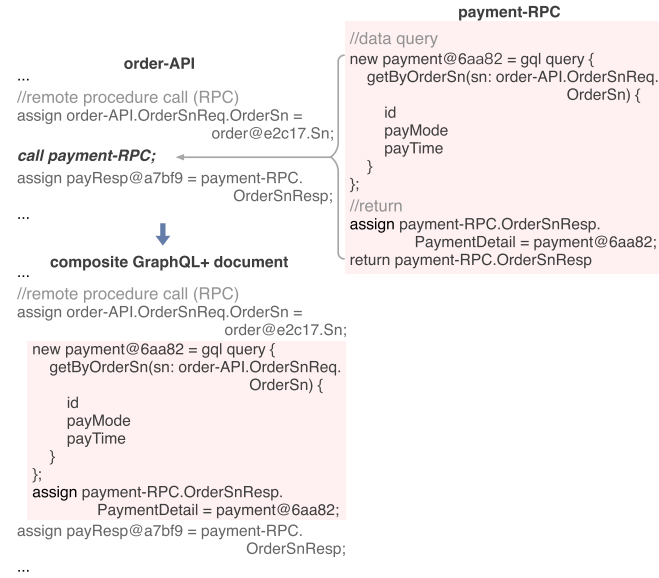


Fig. 4. GraphQL+ Composition.

third-party payment records by invoking the payment-RPC service, thereby returning the order details. Fig. 4 illustrates the respective GraphQL+ documents of the order-API and payment-RPC services. In this scenario, order-API performs a service call to payment-RPC, resulting in inter-service data transmission.

We compose the GraphQL+ documents of individual microservices into a composite document by identifying remote call statements and replacing them with the corresponding GraphQL+ documents of the invoked services. This transforms inter-service communication into local execution. As shown in Fig. 4, the statement `call payment-RPC` in the order-API service is replaced with the GraphQL+ document of payment-RPC.

In order to avoid variable name collisions during the composition, we categorize variables based on their scopes into local variables, which are confined to individual microservices, and global variables, which are used for inter-service communication. For example, when order-API invokes payment-RPC, it first assigns the local variable `order@e2c17.Sn` to the global variable `order-API.OrderSnReq.OrderSn`. The payment-RPC receives this global variable as an input and assigns the execution result to the global variable `payment-RPC.OrderSnResp`. And then, order-API assigns this result to the local variable `payResp@a7bf9` for further processing.

The process of GraphQL+ composition is detailed in Algorithm 1. The algorithm begins by obtaining the GraphQL+ document of the entry service and then identifying remote call statements within it (Lines 1–2). As long as such remote calls remain (Line 3), the algorithm gets the GraphQL+ document of the invoked service (Line 5) and checks for circular dependencies (Lines 6–8). If a dependency is detected, the process terminates with an error; otherwise, the remote call statement is replaced by the GraphQL+ document of the invoked service (Line 9). The revised document is reexamined to identify any remote calls (Line 11). This iterative process continues until

Algorithm 1: GraphQL+ Composition

Input: $QLs = \{QL_i | s_i \in S\}$, where QL_i denotes the GraphQL+ document of the service s_i

Output: composite GraphQL+ document QL

```

1 QL ← getEntry(QLs); //get the GraphQL+ document of
  the entry service
2 CS ← identifyCallStmt(QL); //remote call statements
3 while CS ≠ ∅ do
4   for ∀ cs ∈ CS do
5     nextQL ← getNext(cs); //get the GraphQL+
      document of the invoked service
6     if detectCircularDependency(nextQL) then
7       return CircularDependencyError;
8     end
9     replace(cs, nextQL); //replace the remote call
      statement by the GraphQL+ document of
      invoked service as illustrated in Fig. 4
10  end
11  CS ← identifyCallStmt(QL);
12 end
13 return QL; //the composite GraphQL+ document

```

no remote calls exist (Line 13). In other words, the composite GraphQL+ document eliminates all remote calls, transforming inter-service data transfer into local computation.

Plugin: We decouple the data processing code of microservices from the database queries and network transmission. Specifically, we modularize and encapsulate the data processing code into the executable plugin packages that are uniformly deployed to the same computing node. On this node, the GraphQL+ engine performs unified data processing through local invocation of the plugins. For example, as shown in Fig. 1, the data processing code of the order-API (Lines 4–8) is encapsulated into the plugin package `orderDetail.so`, which provides a `procOrderHandle` interface. Once deployed to the engine, the plugin can be dynamically executed at runtime.

The plugin interfaces use JSON for data exchange, ensuring language independence and interoperability. The interface definitions include standardized input/output handling to guarantee consistent data exchange. This module supports the plugin registration and resource isolation. These capabilities facilitate efficient maintenance of both the plugin repository and runtime state management, ensuring that the system remains stable during upgrades. With resource isolation, the plugins can coexist without interference, and the hot update capability allows for seamless upgrades without service interruption.

Parser: After deploying the composite GraphQL+ document, the parser initiates the syntax analysis process, transforming the document into an Abstract Syntax Tree (AST).

Based on the GraphQL+ specification, we adopt a scannerless recursive-descent parser that performs syntax reduction and character-level matching directly on the input stream. By unifying lexical and syntactic rules, the parser better handles nested and context-sensitive constructs. Each syntactic

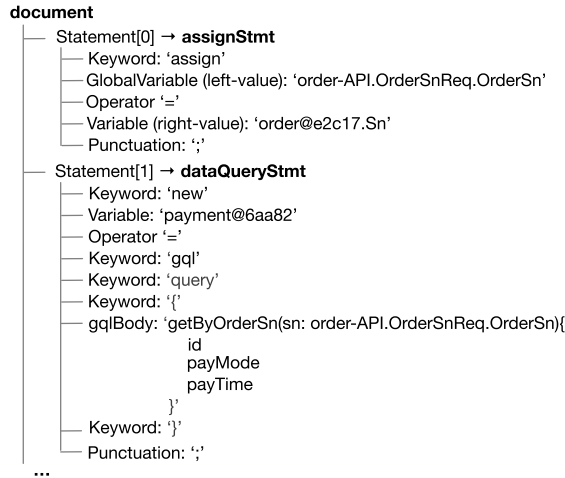


Fig. 5. Abstract syntax tree example.

unit (operation types, field paths, variable references, built-ins) is instantiated as an AST node with recorded source locations (line/column) to support error localization. The resulting hierarchical AST preserves grammatical constructs, semantic boundaries, parent-child nesting, and sequential dependencies, serving as the intermediate representation for execution-plan generation and runtime scheduling.

For example, the AST of a composite GraphQL+ document is shown in Fig. 5. The first subtree corresponds to an assignment statement, where `order-API.OrderSnReq.OrderSn` is a global variable and `order@e2c17.Sn` is a local variable, with an assignment relationship between the two. The second subtree represents a data query statement. Through the syntax analysis, the GraphQL+ document is transformed into an AST, which provides support for the executor.

Executor: The AST nodes contain the semantic information required to execute the operations. The executor traverses the AST recursively, visiting each node step by step. For each node, the executor generates the corresponding execution action and executes the operation accordingly.

For example, through traversing the AST in Fig. 5, the executor performs two actions as shown in Fig. 6. First, the executor assigns the local variable `order@e2c17.Sn` to the global variable `order-API.OrderSnReq.OrderSn`, and executes a standard GraphQL operation, taking the variable `order-API.OrderSnReq.OrderSn` as input. The execution result is stored in the variable `payment@6aa82`.

At runtime, the executor instantiates a context object to serve as the carrier of runtime state. The context object contains the variable pool (storing the variables), execution logs (recording the execution process and debugging information), and an exception collector (error detection), which enhances the executor's observability and fault tolerance. The executor executes actions in sequence, with the execution results being written back to the variable pool, thus managing the lifecycle of variables and enabling seamless execution context.

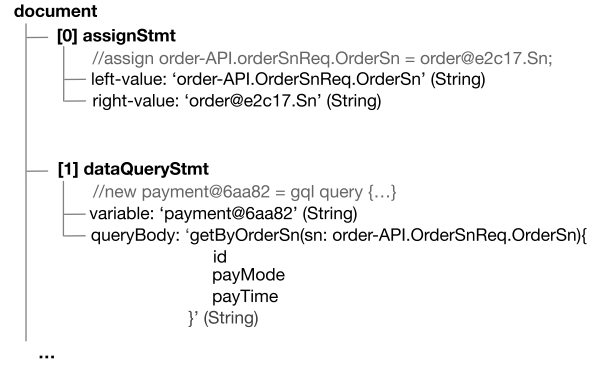


Fig. 6. AST-Based execution actions.

Database Gateway: The gateway provides a unified database access interface for the GraphQL+ engine. Each database registers with the gateway by declaring its data model and access rules. The gateway then creates adapters for each database, enabling support for heterogeneous databases. It manages connections, performs data schema conversions, and enforces access control at the database level. For example, Sequelize, a Node.js ORM (Object-Relational Mapping) library, can be used for interacting with databases. There is an example of a database model based on Sequelize as below.

```
const sequelize = new
Sequelize("postgres://10.77.110.222:5432/order-API-db",{
  dialect: 'postgres',
}); const Order = sequelize.define('Order', {
  id: {
    type: DataTypes.BIGINT,
    primaryKey: true,
  },
  sn: {
    type: DataTypes.STRING,
    unique: true
  },
});
```

The above code defines a database model and configures the connection details for a PostgreSQL database. It declares the table `Order`, including a primary key `id` and a unique `sn` field representing the order number.

To enforce read-only control, a hook function is defined.

```
Order.beforeUpdate((instance, options) => {throw new Error('only read');})
```

The above code registers a hook function for the `Order` model. This function is triggered before update operation is performed on the `Order` table. If triggered, it throws a read error, thereby preventing the update from occurring. It ensures that certain data records cannot be modified at runtime, which strengthens the access control logic at the database level. The hook function implements a unified data access security policy. Moreover, the database gateway supports horizontal scaling behind a load balancer, enabling multiple gateway instances to be deployed as the number of microservice databases grows. Additionally, the gateway now manages security mechanisms across multiple datasets, which were previously handled by individual microservices. While this design may introduce potential risks related to cross-database access, it also creates opportunities for further security enhancement, such as authentication and authorization middleware or policy enforcement modules, by

TABLE I
MICROSERVICE PROJECTS

Scale	Project	Language	RPC	# of services	URL
small	staffjoy	Java	RESTful API/JSON	2	https://github.com/spring2go/staffjoy
	piggymetrics	Java	RESTful API/JSON	3	https://github.com/sqshq/piggymetrics
	novel-cloud	Java	RESTful API/JSON	3	https://github.com/201206030/novel-cloud/
	guli	Java	RESTful API/JSON	3	https://github.com/NiceSeason/gulimall-learning
	go-zero-look	Golang	gRPC/Protobuf	3	https://github.com/Mikaelemmmm/go-zero-looklook
medium	mall-swarm	Java	RESTful API/JSON	4	https://github.com/macrozheng/mall-swarm
	go-zero-mall	Golang	gRPC/Protobuf	4	https://github.com/nivin-studio/go-zero-mall
	train-ticket	Java	RESTful API/JSON	6	https://github.com/FudanSELab/train-ticket
large	ecommerce	Java	RESTful API/JSON	12	https://github.com/hoangtien2k3/ecommerce-microservices

enabling consistent and unified management of database access rules.

In summary, the GraphQL+ engine comprises four modules: Composition, Parser, Executor, and Plugin. The Composition module merges individual microservice GraphQL+ models into a composite model. The Parser converts that composite model into an AST. The Executor generates the execution plan based on the AST, coordinating calls to Plugins, which encapsulate concrete data-processing and transformation logic. Moreover, microservices usually maintain their own databases independently. The database gateway provides a unified interface for accessing these distributed databases. These modules form a coherent pipeline that transforms inter-service data processing and transmission into unified computation.

IV. EVALUATION

To evaluate our approach, we study nine microservice projects: *staffjoy*, *piggymetrics*, *novel-cloud*, *guli*, *go-zero-look*, *go-zero-mall*, *mall-swarm*, *train-ticket*, and *ecommerce*, which are popular open-source projects on GitHub. These projects span Java and Golang, the two prevalent languages in GitHub's microservice repositories (29.6% and 10.9%, respectively)¹. Each project has received at least 300 stars (with an average of about 4.6k), reflecting good community adoption, and features data-intensive workloads, making them representative for evaluation in real-world scenarios. This enables our experimental subjects to reflect the architectural diversity of microservice systems in practical applications.

The detailed information about these projects is provided in Table I. We categorize these projects by the number of services to represent system scale: *small* (2-3 services), *medium* (4-6 services), and *large* (12 services). An increase in the number of services participating in data transmission typically reflects greater service dependency complexity, higher inter-service communication overhead, and more significant performance challenges. Among these projects, seven are written in Java, and two are written in Golang. Microservice projects written in Java typically rely on RESTful APIs, but their performance is insufficient under high load due to their lack of optimization for high concurrency. Additionally, since data is transmitted in the

JSON format, the serialization and deserialization overhead can further impact performance. In contrast, microservice projects written in Golang often use gRPC for communication. gRPC is an efficient, HTTP/2-based remote procedure call protocol that leverages Protocol Buffers (Protobuf) for data serialization. Protobuf is more compact and efficient than JSON, reducing bandwidth usage. In the experiments, for the Golang projects all approaches use gRPC with Protobuf for inter-service communication and data serialization, whereas for the Java projects all approaches use RESTful HTTP with JSON. This setup ensures a fair comparison by keeping the RPC and serialization stack consistent within each project.

We pose the following research questions:

- **RQ1:** *How effective is the LLM-based multi-agent system for GraphQL+ document generation?* In the multi-agent system, an iterative agent workflow is employed to generate GraphQL+ documents. This question investigates the practical feasibility of the iRUC approach, with a particular focus on the number of iterations required to pass functionality and syntax checks.
- **RQ2:** *How does the iRUC approach perform in terms of throughput and latency compared to the existing work on inter-service data communication optimization?* This question focuses on quantifying the performance benefits of the iRUC approach in terms of throughput and latency, benchmarking it against existing optimization approaches for inter-service data communication. The results aim to reveal whether iRUC offers a high-performance solution for data-intensive microservice systems.
- **RQ3:** *How effective is the iRUC approach in reducing the inter-service data communication?* This question focuses on the volume of data transferred within microservice systems. A substantial reduction in data transfer can indicate that iRUC effectively mitigates inter-service communication overhead.

A. Automatic GraphQL+ Generation

Claude 4.5 Sonnet² is a widely used large language model (LLM), known for its coding capabilities. Gemini 2.5 Pro³

¹<https://github.com/topics/microservices>

²<https://www.anthropic.com/claude/sonnet>

³<https://gemini.google.com/app>

TABLE II
LLM AGENTS: TIME CONSUMPTION (SECOND)

Scale	Project	L	M	P	G	C	Total
small	staffjoy	20.1	25.1	34.4	42.6	34.8	157
	piggymetrics	36.0	43.8	84.7	55.8	33.8	254.1
	novel-cloud	25.5	60.0	55.7	35.1	30.7	207
	guli	36.1	41.5	60.2	54.8	24.0	216.6
	go-zero-look	39.5	32.2	44.1	43.9	41.4	201.1
medium	mall-swarm	32.6	45.9	44.6	48.9	33.9	205.9
	go-zero-mall	41.4	116.4	56.4	118.8	32.7	365.7
	train-ticket	27.0	33.3	76.6	47.7	26.7	211.3
large	ecommerce	37.9	45.4	86.3	89.3	31.7	290.6

Abbreviations for LLM Agents in Fig. 2: L = Labeling, M = Modularizing, P = Packaging, G = Generating, C = Functionality Checking.

is a popular LLM delivering competitive performance across diverse reasoning tasks. PEG.js⁴ is a parser generator grounded in Parsing Expression Grammar (PEG), capable of generating the parser that can validate and analyze GraphQL+ syntax. Based on these models, we employ the Claude 4.5 Sonnet API to generate GraphQL+ documents from microservice code through an agent workflow consisting of labeling, modularizing, packaging and generating. Moreover, we use the Gemini 2.5 Pro for functionality check and PEG.js for syntax check. Functionality check ensures that GraphQL+ documents are equivalent to the original code, resetting the workflow on failure. Similarly, syntax check ensures adherence to defined rules, returning to the model regeneration if it fails.

Table II reports the per-round time consumption of LLM agents across projects; the average per-round total time ranges from 157s to 365.7s. Across agents, modularizing (M), packaging (P) and generating (G) dominate the time consumption and show large cross-project variability (approximately 25.1s to 118.8s), indicating that these three agents are the primary contributors to the observed differences in total time between projects. By contrast, labeling (L) and functionality checking (C) are comparatively small and stable, with values roughly in the 20.1s to 41.4s range. Note that after functionality checking, syntax checking is performed using a PEG parser; its average execution time is below 1ms and is negligible compared with that of the LLM-based agents.

Table III presents the number of errors identified during the agent workflow for the GraphQL+ document generation. For the nine microservice projects evaluated, the functionality check is completed within at most 4 iterations, and the syntax check within at most 3 iterations. The *ecommerce* project has the largest number of microservices among all projects. It requires a total of 7 iterations to pass both the functionality and syntax checks (4 iterations for functionality and 3 iterations for syntax). The *staffjoy* project has the fewest number of microservices, and successfully passes all checks within just 3 iterations. Throughout the iterative workflow, both the functionality-checking and syntax-checking agents effectively identify issues, allowing the

TABLE III
FUNCTIONALITY AND SYNTAX CHECKING

Scale	Project	Functionality Errors				Syntax Errors		
		I1	I2	I3	I4	I1	I2	I3
small	staffjoy	0	/	/	/	1	0	/
	piggymetrics	1	0	/	/	2	0	/
	novel-cloud	3	0	/	/	2	0	/
	guli	4	0	/	/	9	1	0
	go-zero-look	0	/	/	/	3	0	/
medium	mall-swarm	2	0	/	/	6	0	/
	go-zero-mall	0	/	/	/	5	4	0
	train-ticket	8	2	0	/	4	0	/
large	ecommerce	11	2	1	0	6	1	0

I1, I2, and so on denote the number of errors found in each iteration.

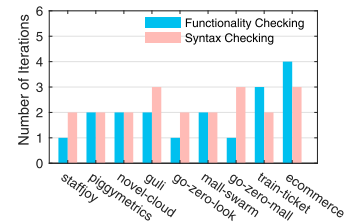


Fig. 7. Number of agent workflow iterations.

number of errors to progressively converge, resulting in correct and executable documents.

Fig. 7 depicts the number of iterations for the agent workflow. On average, functionality checking completes in 2 iterations, while syntax checking requires 2.3 iterations. The maximum observed counts are 4 iterations for functionality checking and 3 iterations for syntax checking.

Moreover, we design 606 test cases for these microservice projects, including normal scenarios (98 cases), boundary conditions (294 cases), and exceptional situations (214 cases), to check whether the generated GraphQL+ documents are functionally consistent with the original microservice code. For these microservice projects, the generated GraphQL+ documents pass all test cases. An example set of the test cases is provided in Appendix B of the supplementary materials.

Answer to RQ1: For these nine microservice projects, the GraphQL+ documents generated by the LLM-based multi-agent system pass all the tests. The results demonstrate that the agent system has the ability to effectively generate the correct and executable GraphQL+ documents from microservice code.

B. Performance Evaluation

We develop a GraphQL+ engine that utilizes PEG.js for syntax parsing and Apollo Server for executing standard GraphQL operations. Using PEG.js, we generate a parser capable of translating GraphQL+ documents into abstract syntax trees (ASTs). Apollo Server⁵, a specification-compliant GraphQL server, is

⁴<https://github.com/pegjs/pegjs>

⁵<https://www.apollographql.com/docs/apollo-server>

responsible for handling data queries and orchestrating interactions with databases. In addition, we develop the database gateway based on Apollo Gateway⁶, serving as the unified entry point for querying databases.

To deploy these microservice projects, we utilize Huawei Elastic Cloud Servers (ECS)⁷, known for their high performance, reliability, and scalability. The deployment involves three types of nodes: database nodes, service nodes, and an engine node for executing GraphQL+ documents. For the service nodes, we select the x1e.4u.8g instances (4vCPUs and 8GB memory), which are distributed across different geographic locations with a network bandwidth of 300Mbps. Following the loosely coupled principle in microservice deployment, the microservices are deployed on different nodes, each with its own independent database, e.g., MySQL and PostgreSQL. To fully leverage database performance, we choose the ac8.2xlarge.2 instances (8vCPUs and 16GB memory) as the database nodes to handle high-performance requirements under heavy loads. The database nodes are hosted in a Huawei private cloud data center. Additionally, we select the ac8.2xlarge.2 instance (8vCPUs and 16GB memory) for the engine node, which is also deployed within the data center and connected to the database nodes via high-bandwidth links (up to 20Gbps).

Compared Approaches. The compared approaches are:

- **MDS** (Microservice-based Data-intensive System): As a baseline approach, discussed in [4], the microservices directly interact with the database to fetch data, then pass the data to downstream services;
- **DD** (Dynamic Deployment) [13]: The microservices are deployed within the same data center to reduce their network distance. In our experiments, all services are relocated to the engine node within the data center;
- **SC** (Serverless Computing) [23]: The microservices run as lightweight functions on a serverless platform; It leverages data locality and network metrics to dynamically migrate services to the same host or nearby nodes, reducing inter-service communication overhead; We deploy the microservice projects on Alibaba Function Compute⁸, a leading serverless computing platform in China;
- **ER** (Excessive data transfer Removal) [17]: This approach inspects the excessive data not utilized within microservices nor transmitted onward, and eliminates these data from network transmission;
- **iRUC** (inter-service data communication Reduction via Unified Computation): Our approach significantly enhances system performance through unified computation.

Performance Metrics. Apache JMeter is a performance testing framework and has been widely used as a testing tool for Web applications. We use JMeter to simulate request loads and apply the following metrics, which are critical for data-intensive systems, to analyze the performance.

- **Throughput** represents the system's capacity to handle requests, quantified in responses per second (*rps*), reflecting the rate at which the system can respond to requests.
- **Latency** denotes the average time to complete a request. For iRUC, it comprises the *service time* (parsing and executing the GraphQL+ model, plugin processing, and database gateway operations) and the *data transfer time* (between the engine and database). For the baseline approaches, the latency includes the *service time* (service execution and database access) and the *data transfer time* (between microservices).

We evaluate the impact of increasing data size per request, ranging from 0.5MB to 6MB. For each microservice project, we design multiple categories of workloads that cover normal scenarios, boundary conditions, and exceptional situations. In the experiments, 95% of the requests are randomly sampled from normal cases, while the remaining 5% are drawn from exceptional cases that simulate anomalous conditions. Each experiment maintains a constant intensity of 250 requests per second for a duration of 120 seconds per run. Fig. 8(a) shows the experimental results for the small-scale microservice projects. For the MDS approach, throughput drops sharply from 74.1*rps* to 6.2*rps* and latency rises from 0.459s to 7.194s, with data transfer time dominating 78% of the total latency. The ER approach, which eliminates unnecessary data transmissions, reduces the communication overhead, yet throughput still falls to 6.6*rps* and latency rises to 6.185s. The DD approach co-locates microservices on the same node, eliminating the network distance: at 0.5MB, throughput reaches 178.8*rps* and latency is 0.075s, and even at 6MB, throughput remains 39.3*rps* with 1.073s latency. The SC approach underperforms DD for small volumes but matches DD at 3.5MB, as it dynamically migrates microservices to the same or adjacent nodes. The iRUC approach, using the GraphQL+ engine to handle the data processing, achieves the most stable performance, with throughput from 182.7*rps* to 80.1*rps* and latency from 0.068s to 0.416s. The breakdown analysis indicates the data transfer time is the dominant factor in latency, which accounts for 78.0%, 76.9% and 60.6% of the total latency in the MDS, ER and DD approaches, respectively. The improvement of the iRUC approach is due to that the data transfer time is only 1.95%, 4.38%, and 37.5% of those in the MDS, ER and DD approaches, respectively.

Fig. 8(b) and 8(c) show the experimental results for the medium-scale and large-scale microservice projects, respectively. We observe the results similar to those for the small-scale projects: the iRUC approach outperforms the MDS, ER, DD, and SC approaches across all experiments. We also observe that performance for all approaches degrades as system scale increases. As the scale increases from small to large, on average, iRUC's throughput decreases from 130.2*rps* to 82.1*rps*, and its latency only increases from 0.202s to 0.539s. The performance of MDS, ER, DD, and SC also declines significantly; for MDS, throughput falls to 12.5*rps* and latency rises to 4.662s. These results indicate that iRUC exhibits better scalability.

Table IV shows the performance improvements of the iRUC approach compared to the MDS, ER, DD and SC approaches.

⁶<https://www.apollographql.com/docs/federation/v1/gateway>

⁷Huawei Cloud, <https://www.huaweicloud.com/intl/en-us/product/ecs.html>

⁸<https://www.alibabacloud.com/en/product/function-compute>

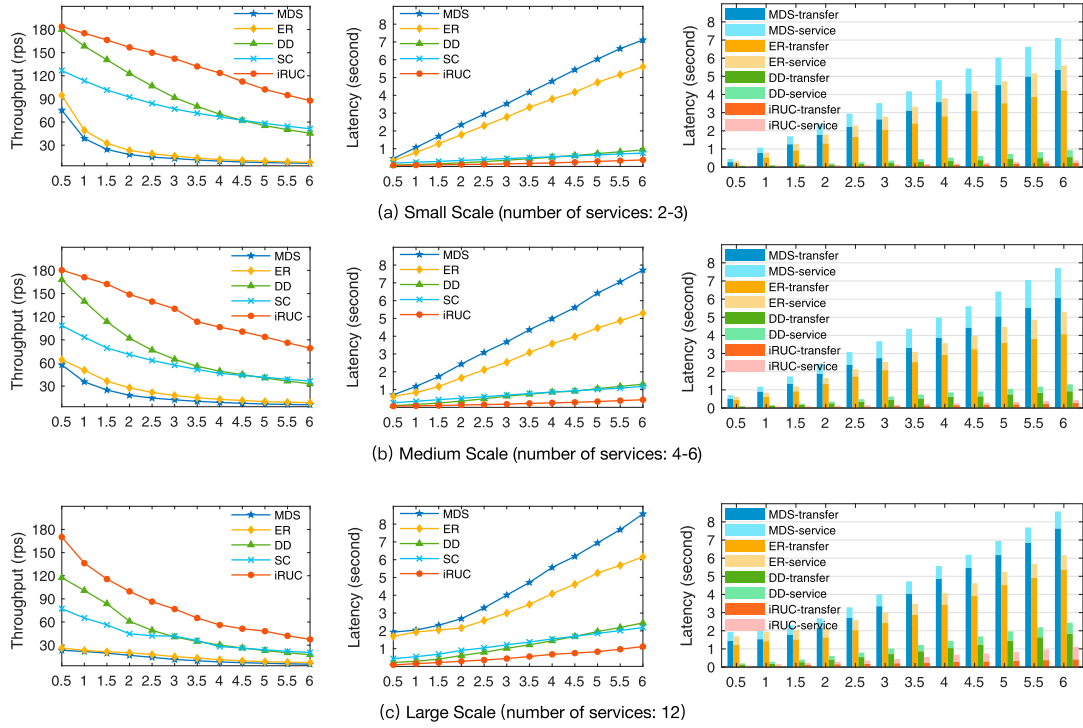


Fig. 8. Impact of data size per request (from 0.5MB to 6MB).

TABLE IV
PERFORMANCE IMPROVEMENT

Project	Throughput (responses per second)									Latency (second)								
	MDS	ER	DD	SC	iRUC	iRUC MDS	iRUC ER	iRUC DD	iRUC SC	MDS	ER	DD	SC	iRUC	iRUC MDS	iRUC ER	iRUC DD	iRUC SC
staffjoy	21.5	30.4	112.2	92.3	149.3	6.94×	4.91×	1.24×	1.62×	3.507	2.376	0.27	0.369	0.182	4.1%	6.1%	53.4%	39.2%
piggymetrics	18.5	21.6	91.6	76.0	128.7	6.95×	5.95×	1.4×	1.69×	4.074	3.102	0.487	0.505	0.212	5.2%	6.8%	43.5%	42.0%
novel-cloud	18.9	24.4	92.0	76.5	134.0	7.09×	5.49×	1.46×	1.75×	3.893	3.015	0.471	0.503	0.187	4.8%	6.2%	39.6%	37.1%
guli	18.7	18.7	87.4	74.5	128.0	6.83×	6.83×	1.46×	1.72×	3.894	3.894	0.526	0.517	0.209	5.4%	5.4%	39.8%	40.5%
go-zero-look	19.3	27.4	93.6	80.6	138.2	7.17×	5.04×	1.48×	1.71×	3.879	2.641	0.428	0.46	0.173	4.5%	6.6%	40.5%	37.6%
mall-swarm	17.6	23.7	86.7	64.1	132	7.51×	5.56×	1.52×	2.06×	4.0	2.746	0.55	0.66	0.194	4.8%	7.1%	35.2%	29.4%
go-zero-mall	19.0	29.6	71.9	69.5	130.7	6.86×	4.42×	1.82×	1.88×	3.967	2.286	0.685	0.572	0.198	5.0%	8.7%	40.0%	34.6%
train-ticket	15.4	17.9	70.1	49.2	115.3	7.51×	6.46×	1.65×	2.34×	4.277	3.524	0.774	0.942	0.27	6.3%	7.7%	34.6%	28.9%
ecommerce	12.5	15.1	50.4	40.3	82.1	6.58×	5.43×	1.63×	2.04×	4.662	3.559	1.191	1.293	0.539	11.6%	15.1%	45.2%	41.7%
Average	17.9	23.2	84.0	69.2	126.5	7.05×	5.57×	1.52×	1.87×	4.017	3.016	0.598	0.647	0.24	5.7%	7.7%	40.7%	37.4%

For these microservice projects, the throughput of the iRUC approach is at least $6\times$ that of the MDS approach, $4\times$ that of the ER approach, $1.2\times$ that of the DD approach and $1.6\times$ that of the SC approach. On average, iRUC achieves $7.05\times$, $5.57\times$, $1.52\times$, and $1.87\times$ higher throughput compared with MDS, ER, DD, and SC, respectively. In terms of latency, iRUC reduces latency to 5.7%, 7.7%, 40.7%, and 37.4% of the latency for MDS, ER, DD, and SC, respectively.

We evaluate the impact of workload intensity on system performance using an ecommerce project comprising 12 microservices, with an average data size of 3MB per request. The workload intensity increases from 1 request/min, 1 request/s to 350 requests/s. Each run lasts 120 seconds, except for the 1 request/min setting, for which the duration is extended to 10 minutes. As shown in Fig. 9, MDS and ER become

saturated at a workload intensity of 150 requests/s, achieving a peak throughput of only about 15rps. As the load increases further, their average latency rises sharply to approximately 5s, indicating that both approaches are constrained by inter-service data-transfer bottlenecks. DD and SC perform better than MDS and ER, achieving a higher throughput of around 40rps; however, under high concurrency, their average latency still increases to about 1.5s, suggesting that although they shorten network transmission paths, they fail to eliminate the overhead of inter-service communication. In contrast, iRUC consistently outperforms all other approaches: its throughput continues to increase with the workload, reaching a peak of 76.4rps, while maintaining a low average latency (below 0.7s), demonstrating superior scalability and resilience to performance saturation. These results confirm that iRUC achieves

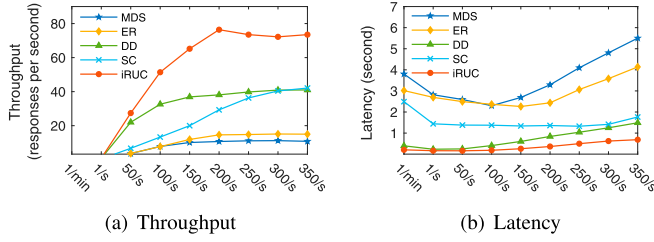


Fig. 9. Impact of workload intensity (from 1 req/min, 1 req/s to 350 req/s).

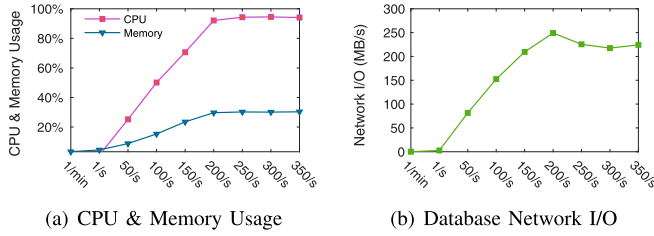


Fig. 10. Resource consumption of GraphQL+ engine.

higher throughput and more stable latency in data-intensive environments.

iRUC adopts a GraphQL+ engine to perform cross-service data processing centrally, eliminating the overhead caused by inter-service data communication, while database access over the network remains necessary. As shown in Fig. 10, for a GraphQL+ engine, CPU utilization increases nearly linearly with load and saturates at about 95% around 250 requests/s, while memory utilization remains low (about 30%) at medium to high loads. The network I/O between the engine and the databases also increases with load and peaks at 249.2 MB/s; once CPU saturation is reached, throughput no longer increases and network I/O stabilizes at 220–250 MB/s. Since the engine can be deployed close to the databases and connected via high-bandwidth links (up to 20 Gbps in the experiments), network I/O does not become the bottleneck. Under this configuration, higher throughput can be achieved by horizontally scaling the GraphQL+ engine behind a load balancer.

Answer to RQ2: The iRUC approach significantly outperforms the MDS, ER, DD and SC approaches in both throughput and latency across all experiments. The experimental results demonstrate that iRUC provides a high-performance solution for data-intensive microservice systems.

Table V presents the data transfer per request, representing the end-to-end volume including both database network I/O and inter-service data communication, under different system scales, as well as the reduction ratios of iRUC compared to the baseline approaches. The DD approach deploys microservices on the same node to eliminate network distance, but data transfer between microservices still occurs. The data transfer volume of DD remains approximately the same as that of MDS. Note that the data-transfer volume per request for SC is not measured due to the deployment on the Alibaba Function Compute, where data transfer statistics are not accessible. Since SC (like DD) shortens network distance rather

TABLE V
DATA TRANSFER REDUCTION

Approach	Data Transfer per Request (MB)			Reduction Ratio (1 – iRUC/baseline)		
	small	medium	large	small	medium	large
MDS	12.58	18.8	32.63	74.3%	82.7%	89.2%
ER	10.45	14.1	25.02	69.0%	77.0%	85.9%
DD	12.25	18.9	32.58	73.6%	82.8%	89.2%
iRUC	3.23	3.25	3.53	/	/	/

than reducing data volume, and they have comparable throughput and latency, SC's data-transfer volume is similar to that of MDS and DD. The ER approach eliminates unnecessary data exchanges between microservices, leading to a lower data transfer volume compared with both MDS and DD. The iRUC approach consistently maintains the average data transfer per request within the range of 3.23–3.53 MB, which is significantly lower than that of the baseline approaches MDS, ER, and DD. Taking MDS as an example, iRUC reduces the data transfer by approximately 9.35 MB, 15.55 MB, and 29.1 MB under small, medium, and large scales, corresponding to reduction ratios of 74.3%, 82.7%, and 89.2%, respectively. Overall, as the system scale increases, the data transfer of iRUC exhibits only a slight growth, demonstrating its stability in transfer optimization.

Answer to RQ3: The iRUC approach significantly reduces the inter-service data communication through unified computation. Experimental results demonstrate its effectiveness in mitigating inter-service communication overhead, highlighting its suitability for data-intensive microservice systems.

V. RELATED WORK

A. Inter-Microservice Data Communication

Inter-microservice data communication often incurs significant runtime overhead [5], [6], resulting in low throughput and high bandwidth consumption in data-intensive microservice systems [7], [8]. Existing research has explored three main optimization strategies.

The first strategy is the design-phase decomposition strategy to minimize inter-service communication. It decomposes monolithic systems into microservice candidates using metrics such as coupling, cohesion, and network cost [10]. For instance, Li et al. [9] model the monolithic system as a graph and treat the problem as a clustering problem on such a graph. However, determining the optimal microservices granularity remains challenging [15], as runtime call graphs can be highly dynamic and heavy-tailed [24].

The second strategy is the deployment-phase strategy which improves performance by reducing the distance between services, data, and end users [14], [25]. Lv et al. [13] apply deep Q-learning to optimize service placement at the edge, while data replication techniques cache data to reduce latency and bandwidth use [26]. Furthermore, serverless computing [27] has emerged as a popular dynamic deployment strategy that enables

developers to build, deploy, and run applications in the cloud without the need to allocate or manage virtualized servers and resources, or handle other operational concerns [23]. In other words, it can dynamically offload and migrate microservices at runtime by trading off processing time and communication overhead to achieve better overall performance. Yet, in practice, this strategy can result in deploying numerous interdependent services on shared servers, compromising scalability and loose coupling [15].

The third strategy is the dynamic resource optimization which enhances performance by allocating computational or network resources based on predictive models [11], [12]. For example, Sinan [12] applies scalable machine learning models to allocate resources and maintain latency targets.

Nevertheless, these strategies often assume that all inter-service communications are necessary, an assumption challenged by tools such as MicroProf [17], which found that redundant data transfers account for a significant portion of the communication overhead.

Therefore, taking a step further, our study aims to avoid inter-service data communications that do not require network transmission. We introduce a unified computational approach that abstracts inter-service data processing and transmission into declarative GraphQL+ documents, minimizing inter-service data exchanges. Notably, our approach is complementary to existing strategies, and combining it with existing strategies would further improve the overall performance.

B. Leveraging LLMs for Software Development

Large Language Models (LLMs) have shown strong potential across the software development lifecycle, including requirements engineering, design, coding, quality assurance, maintenance, and management [20], [21]. In requirements engineering, Ezzini et al. [28] demonstrated how ChatGPT improves requirement clarity. In coding, tools such as Codex, CodeGen, InCoder, Copilot, and CodeGeeX [20] assist with generation, completion, summarization, and comprehension. LLMs also enhance software quality through vulnerability detection [29], test automation [30], and program repair for software maintenance [31].

Beyond individual tasks, recent research enhances LLM capabilities by integrating decision-making and interactive problem-solving functions, transforming them into LLM-based agents [20], [21] to enhance software development. Frameworks such as AlpacaFarm [32], SWE-Agent [33], and AutoCodeRover [34] show that agent-based LLM systems can coordinate complex engineering workflows. Furthermore, multi-agent systems such as ChatDev [19] and Self-Collaboration [35] simulate team-based software development following waterfall models [36], while MetaGPT [37] extends this to agile paradigms through standardized operating procedures.

Moving towards domain-specific applications, some studies have customized LLMs for tasks in microservice-based software systems [38]. Huang et al. [39] used LLMs to extract

API entities and relations from unstructured data, and Sarda et al. [40] employed them for auto-remediation by generating playbooks to improve reliability. However, microservice refactoring, a continuous “code-and-fix” process essential for practice, remains underexplored [38].

Hence, in this study, we develop an LLM-based multi-agent system for microservice refactoring that automatically extracts and reconstructs existing systems to reduce inter-service data communication overhead. To our knowledge, this is the first such approach, and experiments on real-world microservice projects validate its effectiveness.

VI. DISCUSSION

A. Threats to Validity

1) *Internal Validity*: The main internal validity threat lies in the selection of microservice systems used in our experiments. To mitigate this, we adopted predefined criteria to select nine widely recognized open-source projects from GitHub rather than developing prototypes. The selection followed three principles: (1) language prevalence: each project is implemented in Java or Go, the most common languages under GitHub’s “microservices” topic; (2) community adoption: each project has at least 300 stars, with an average of about 4.6k stars, reflecting broad interest and real-world usage; and (3) data-intensive workload: each involves substantial data processing and transmission, aligning with our study’s focus. We acknowledge that relaxing these criteria to include additional projects could further validate the generalizability of our findings. A second potential threat concerns the choice of hyperparameters for the Claude 4.5 Sonnet and Gemini 2.5 Pro APIs. To ensure reproducibility and reflect typical usage scenarios, we used the default configurations rather than tuning parameters for specific optimizations.

2) *External Validity*: The external validity concerns the generalization of our findings across different contexts. The primary threat arises from the potential variability in responses from the Claude 4.5 Sonnet and Gemini 2.5 Pro APIs between runs. To mitigate this, we developed our approach iteratively and designed a set of test cases to ensure consistency and correctness between the generated models and the original microservice code. Further studies using other LLMs and investigating the potential impacts from different versions of the generated code would be valuable. Another potential threat stems from network instability during deployment on real-world environments, which could affect performance measurement. To reduce this risk, we conducted evaluations across nine microservice projects, and results consistently showed improved throughput and reduced latency, confirming the robustness of our approach.

3) *Construct Validity*: The construct validity concerns whether the measurements and operational definitions used in our study accurately capture the intended concepts. In our experiments, we measured throughput, the system’s capability to handle requests quantified in responses per second, and latency, the average duration required to complete a request from start to end, as indicators of system performance.

Although these metrics are commonly used and critical for data-intensive systems, they may not fully represent the performance of the whole system. To mitigate this risk, we adopted standardized quantitative indicators and performed repeated experiments to ensure consistency.

4) *Conclusion Validity*: The conclusion validity relates to the extent to which the evidence supports the conclusions drawn. Our experiments were conducted in runtime environments where network conditions and resource scheduling may introduce uncontrolled variance. To minimize this effect, each experiment was executed multiple times under controlled conditions across different projects, and average values were reported. While specific architectural or workload variations may yield different absolute results, the observed trends consistently validate the advantage of our proposed approach in improving throughput and reducing latency.

B. Implications for Future Researchers

With the growing adoption of microservice architectures in data-intensive systems, efficient inter-service communication has become critical for maintaining system performance. This study introduces GraphQL+, an executable declarative language that extends GraphQL with service-invocation and plugin-execution semantics, enabling unified cross-language modeling of data processing and transmission. In addition, we develop an LLM-based multi-agent system that automatically parses existing microservice code to synthesize corresponding GraphQL+ models, minimizing manual refactoring efforts. Although this work effectively reduces inter-service data communication overhead, it is designed to complement, rather than replace, existing optimization strategies. Future research may explore hybrid approaches that integrate GraphQL+ with current design-, deployment-, and resource-optimization techniques to further enhance performance. Additionally, although security is beyond the scope of this study, our approach unifies security management across multiple datasets previously handled by individual microservices. Future work may explore advanced mechanisms, such as authentication and authorization middleware or policy enforcement modules, to further enhance the system's overall security.

Moreover, we validated our approach using open-source microservice systems implemented in Java and Go. Extending the LLM-based multi-agent framework to additional languages such as Python and JavaScript (e.g., React-based systems) could broaden its applicability. Furthermore, as the current system emphasizes correctness in GraphQL+ model generation over refactoring efficiency, future studies may improve performance through LLM fine-tuning or by adopting advanced coordination protocols to enhance agent collaboration and communication efficiency.

C. Implications for Practices

This work offers several implications for service engineers as well as software vendors. For engineers, the proposed approach provides a systematic method to eliminate redundant inter-service data transfers during runtime, thereby improving

throughput and latency without requiring architectural redesign or additional resources. Particularly, our approach provides flexibility that benefits engineers both when they start fresh (by directly designing with GraphQL+) and when they want to migrate existing services (via the LLM-driven reconstruction).

For software vendors, our findings highlight the importance of reducing unnecessary inter-service data communication in existing data-intensive microservice systems to enhance the system's performance, which is somehow overlooked but highly critical for data-intensive systems.

VII. CONCLUSION

This paper proposes the iRUC approach, a unified computation framework designed to address inter-service data communication bottlenecks in microservice-based, data-intensive systems. Specifically, we introduce GraphQL+, an executable, declarative language for unified, cross-language modeling of data processing and transmission across microservices. To automate the construction of GraphQL+ models, we develop an LLM-based multi-agent system that parses existing microservice code and consistently synthesizes the corresponding GraphQL+ models. Furthermore, we implement an engine for central execution of the generated GraphQL+ models, thereby effectively eliminating data transfers between microservices. Experiments on nine GitHub microservice projects deployed in real world environments demonstrate that iRUC significantly reduces data transfers and improves performance: On average, iRUC achieves $5.57\times$ the throughput of the excessive data-transfer removal approach (ER) and reduces latency to 7.7% of that of ER; it achieves $1.52\times$ the throughput of the dynamic deployment approach (DD) and reduces latency to 40.7% of that of DD; and it achieves $1.87\times$ the throughput of the serverless computing approach (SC) and reduces latency to 37.4% of that of SC.

REFERENCES

- [1] C. P. Chen and C.-Y. Zhang, "Data-intensive applications, challenges, techniques and technologies: A survey on big data," *Inf. Sci.*, vol. 275, pp. 314–347, Aug. 2014.
- [2] A. Davoudian and M. Liu, "Big data systems: A software engineering perspective," *ACM Comput. Surveys (CSUR)*, vol. 53, no. 5, pp. 1–39, 2020.
- [3] S. Dittmer et al., "Navigating the development challenges in creating complex data systems," *Nature Mach. Intell.*, pp. 1–6, 2023.
- [4] O. Aitlmoudden, M. Housni, N. Safeh, and A. Namir, "A microservices-based framework for scalable data analysis in agriculture with IoT integration," *Int. J. Interactive Mobile Technol.*, vol. 17, no. 19, Oct. 2023.
- [5] Y. Gan et al., "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proc. 24th Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2019, pp. 3–18.
- [6] N. Lazarev, N. Adit, S. Xiang, Z. Zhang, and C. Delimitrou, "Dagger: Towards efficient RPCs in cloud microservices with near-memory reconfigurable NICs," *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 134–138, Jul.–Dec. 2020.
- [7] P. Ranganathan, "From microprocessors to nanostores: Rethinking data-centric systems," *Computer*, vol. 44, no. 1, pp. 39–48, Jan. 2011.
- [8] V.-N. Pham, M. D. Hossain, G.-W. Lee, and E.-N. Huh, "Efficient data delivery scheme for large-scale microservices in distributed cloud environment," *Appl. Sci.*, vol. 13, no. 2, 2023, Art. no. 886.

- [9] J. Li, H. Xu, X. Xu, and Z. Wang, "A novel method for identifying microservices by considering quality expectations and deployment constraints," *Int. J. Softw. Eng. Knowl. Eng.*, vol. 32, no. 3, pp. 417–437, 2022.
- [10] Y. Abgaz et al., "Decomposition of monolith applications into microservices architectures: A systematic review," *IEEE Trans. Softw. Eng.*, vol. 49, no. 8, pp. 4213–4242, Aug. 2023.
- [11] B. Urgaonkar, P. Shenoy, A. Chandra, and P. Goyal, "Dynamic provisioning of multi-tier internet applications," in *Proc. 2nd Int. Conf. Autonomic Comput. (ICAC)*, Piscataway, NJ, USA: IEEE Press, 2005, pp. 217–228.
- [12] Y. Zhang, W. Hua, Z. Zhou, G. E. Suh, and C. Delimitrou, "Sinan: ML-based and QoS-aware resource management for cloud microservices," in *Proc. 26th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2021, pp. 167–181.
- [13] W. Lv et al., "Microservice deployment in edge computing based on deep q learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2968–2978, Nov. 2022.
- [14] R. P. Ishii and R. F. de Mello, "An adaptive and historical approach to optimize data access in grid computing environments," *INFOCOMP J. Comput. Sci.*, vol. 10, no. 2, pp. 26–43, 2011.
- [15] M. Hamza, M. A. Akbar, K. Smolander, and A. A. Khan, "Practitioners' perspective on microservices design areas challenges: A socio-technical grounded theory literature review," in *Proc. Annu. Symp. Comput. Sci.*, 2023, pp. 1–10.
- [16] P. Genfer and U. Zdun, "Avoiding excessive data exposure through microservice APIs," in *Proc. Eur. Conf. Softw. Archit.*, New York, NY, USA: Springer-Verlag, 2022, pp. 3–18.
- [17] S. S. M. Tariq, L. Menard, P. Su, and P. Roy, "MicroProf: Code-level attribution of unnecessary data transfer in microservice applications," *ACM Trans. Archit. Code Optim.*, vol. 20, no. 4, pp. 1–26, 2023.
- [18] GraphQL Working Draft. Accessed: Feb. 25, 2026. [Online]. Available: <https://spec.graphql.org/draft/>
- [19] C. Qian et al., "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2024, pp. 15174–15186.
- [20] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–30, May 2024.
- [21] X. Hou et al., "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, 2024.
- [22] Ford, B. "Parsing expression grammars: a recognition-based syntactic foundation," in *Proc. ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang.*, 2004, pp. 111–122.
- [23] G. R. Russo, V. Cardellini, and F. L. Presti, "A framework for offloading and migration of serverless functions in the edge-cloud continuum," *Pervasive Mob. Comput.*, vol. 100, May 2024, Art. no. 101915.
- [24] S. Luo et al., "Characterizing microservice dependency and performance: Alibaba trace analysis," in *Proc. ACM Symp. Cloud Comput.*, 2021, pp. 412–426.
- [25] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 4th Quart. 2017.
- [26] Y. Liu et al., "QoE-aware data caching optimization with budget in edge computing," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 324–334.
- [27] S. Kounev et al., "Serverless computing: What it is, and what it is not?" *Commun. ACM*, vol. 66, no. 9, pp. 80–92, 2023.
- [28] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh, "Automated handling of anaphoric ambiguity in requirements: a multi-solution study," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 187–199.
- [29] W. Tang, M. Tang, M. Ban, Z. Zhao, and M. Feng, "CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection," *J. Syst. Softw.*, vol. 199, May 2023, Art. no. 111623.
- [30] I. Hayet, A. Scott, and M. d'Amorim, "ChatAssert: LLM-based test oracle generation with external tools assistance," *IEEE Trans. Softw. Eng.*, vol. 51, no. 1, pp. 305–319, Jan. 2025.
- [31] G. De Vito, S. Di Martino, F. Ferrucci, C. Gravino, and F. Palomba, "LLM-based automation of COSMIC functional size measurement from use cases," *IEEE Trans. Softw. Eng.*, vol. 51, no. 5, pp. 1500–1523, May 2025.
- [32] Y. Dubois et al., "AlpacaFarm: A simulation framework for methods that learn from human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 30039–30069.
- [33] J. Yang et al., "Swe-agent: Agent-computer interfaces enable automated software engineering," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 37, pp. 50528–50652.
- [34] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2024, pp. 1592–1604.
- [35] Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-collaboration code generation via ChatGPT," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 7, pp. 1–38, 2024.
- [36] W. W. Royce, "Managing the development of large software systems: concepts and techniques," in *Proc. 9th Int. Conf. Softw. Eng.*, 1987, pp. 328–338.
- [37] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 1–27.
- [38] C. Zhong, S. Li, H. Zhang, H. Huang, L. Yang, and Y. Cai, "Refactoring microservices to microservices in support of evolutionary design," *IEEE Trans. Softw. Eng.*, vol. 51, no. 2, pp. 484–502, Feb. 2025.
- [39] Q. Huang, Y. Sun, Z. Xing, M. Yu, X. Xu, and Q. Lu, "Api entity and relation joint extraction from text via dynamic prompt-tuned language model," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 1, pp. 1–25, 2023.
- [40] K. Sarda, Z. Namrud, M. Litoiu, L. Shwartz, and I. Watts, "Leveraging large language models for the auto-remediation of microservice applications: An experimental study," in *Proc. Companion 32nd ACM Int. Conf. Found. Softw. Eng.*, 2024, pp. 358–369.



Paper Award.



Puwei Wang received the Ph.D. degree in computer science from the Institute of Computing Technology, Chinese Academy of Sciences. He is currently an Associate Professor with the School of Information, Renmin University of China. He has published more than 30 papers in well-known international journals and conferences, such as TSC, TPDS, TKDE, ICSE, ICWS, and ICSOC. His research interests include services computing and blockchain. He received the ICWS 2023 Best Paper Award and the DIKW 2021 Outstanding

Ruiheng Liu received the master degree in computer science from the School of Information, Renmin University of China, in 2025. His research interests include microservice optimization and LLM-based multi-agent systems.



ecosystem and LLM-driven multi-agent system.

Keman Huang received the Ph.D. degree in control science from Tsinghua University. He is currently an Associate Professor with the School of Information, Renmin University of China. He has published more than 60 papers in well-known international journals and conferences, such as IEEE TRANSACTIONS ON SERVICES COMPUTING, IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING, IJIM, MIT SMR, IEEE ICWS, ACM CSCW, ICSOC and AAAI etc. His research interests include data



Xiaoyong Du is currently a Professor with the School of Information, Renmin University of China, specializing in database systems and data engineering. He is also the Director of Key Laboratory of Data Engineering and Knowledge Engineering. He has published more than 200 papers in well-known international journals and conferences, such as SIGMOD, VLDB, TKDE, KDD, AAAI etc.